Oficina Mecânica — Microsserviços e Saga Pattern

POS TECH · Software Architecture · Tech Challenge — Fase 4

1. Identificação

Participante	RM	Contato
Taynan Konrath	RM a preencher	taynan@lara.app.br

2. Links

Vídeo de demonstração (até 15 min)

link do vídeo a preencher

Repositórios dos microsserviços

Microsserviço	Repositório	Banco de dados
OS Service (orquestrador da Saga)	github.com/konrath9/OficinaMecanica	PostgreSQL (relacional)
Billing Service	github.com/konrath9/oficina-billing-service	PostgreSQL (relacional)
Execution Service	github.com/konrath9/oficina-execution-service	MongoDB (NoSQL)

Repositórios de infraestrutura e autenticação (fases anteriores, reaproveitados)

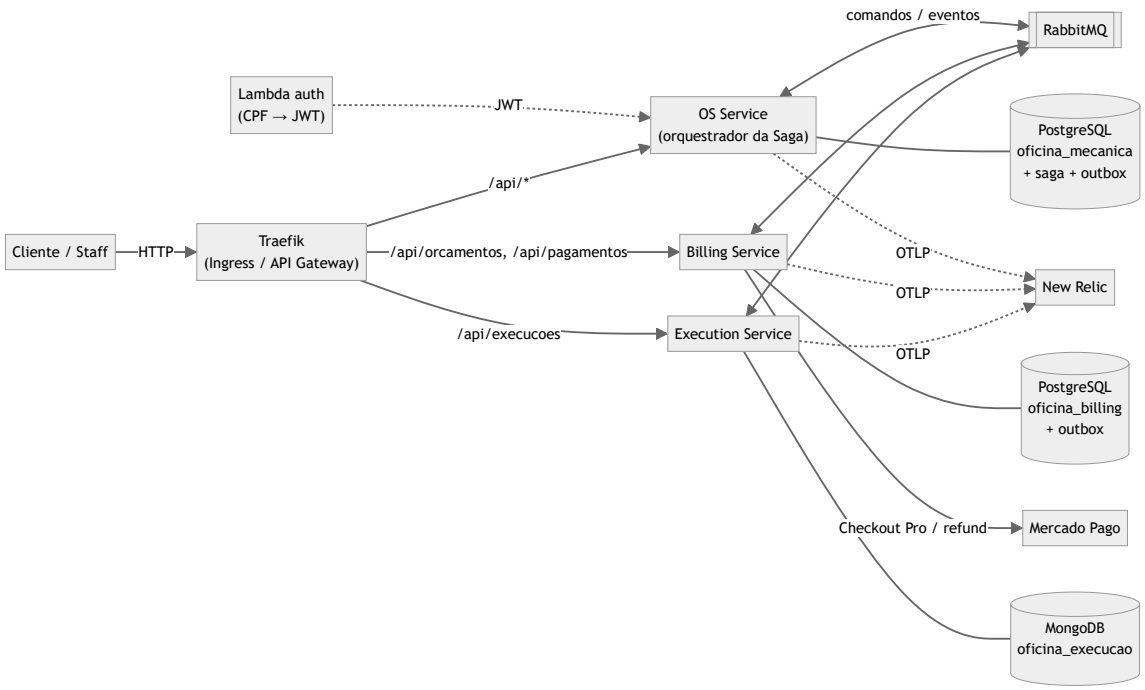
Repositório	Conteúdo
oficina-mecanica-infra-k8s	Terraform do cluster Kubernetes (k3s em EC2 t3.medium, AWS)
oficina-mecanica-infra-db	Terraform da VPC e do RDS PostgreSQL
oficina-mecanica-auth	Lambda de autenticação por CPF (emite o JWT) + API Gateway

Documentação técnica (no repositório do OS Service)

- README, seção Fase 4: github.com/konrath9/OficinaMecanica
- Contratos de mensageria: docs/fase4/contratos-mensageria.md
- ADR da Saga: docs/adr/0005-saga-orquestrada-com-masstransit.md
- Swagger de cada serviço: `/swagger` (OS), `/billing/swagger`, `/execucao/swagger`

3. Arquitetura geral

Três microsserviços independentes, cada um com repositório, banco, pipeline de CI/CD e deploy no Kubernetes próprios. Nenhum serviço acessa o banco de outro: o cliente fala com os serviços por REST (síncrono, pelo Traefik), e entre eles a comunicação é assíncrona, por mensagens no RabbitMQ.



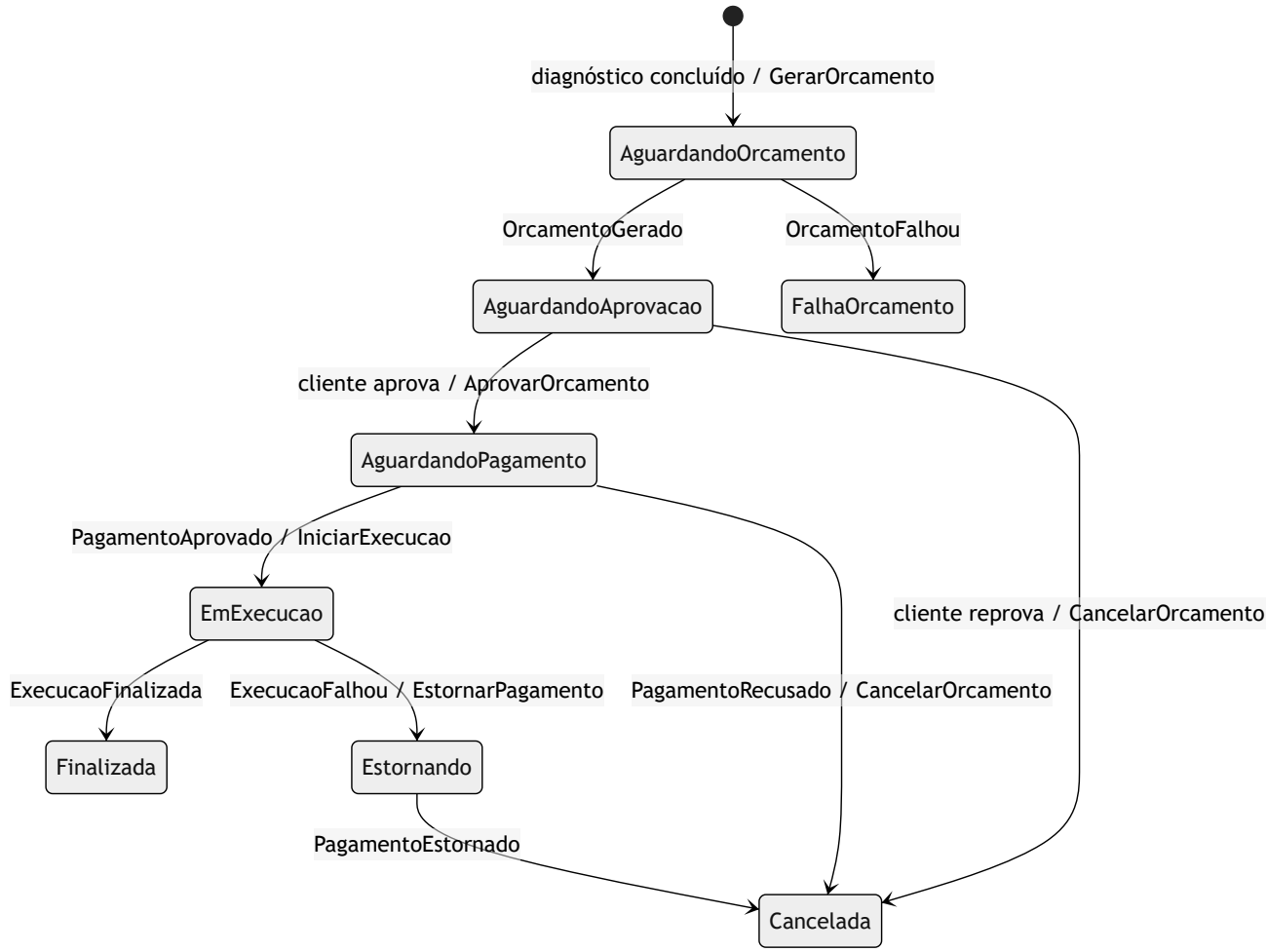
Serviço	Responsabilidade	Expõe / consome
OS Service	Abertura da OS, status e histórico, clientes, veículos, catálogo de serviços e peças, login de staff. Orquestra a Saga e notifica o cliente por e-mail a cada mudança de status.	REST para cliente e staff; publica comandos e consome eventos da Saga
Billing Service	Geração do orçamento, cobrança via Mercado Pago (Checkout Pro, webhook validado na API do MP), estorno.	REST de consulta, simulação e webhook; consome comandos de orçamento e pagamento
Execution Service	Fila de execução priorizada, diagnóstico e reparo serviço a serviço, finalização ou registro de falha.	REST para o mecânico; consome comandos de execução

Infraestrutura

- **Kubernetes**: k3s na AWS (EC2 t3.medium, provisionado por Terraform), namespace único, Traefik como ingress, HPA nas APIs.
- **Mensageria**: RabbitMQ no próprio cluster; MassTransit 8 nos três serviços.
- **Bancos**: RDS PostgreSQL com um database por serviço relacional (`oficina_mecanica`, `oficina_billing`) e MongoDB em StatefulSet para o Execution.
- **Observabilidade** (a mesma da Fase 3): OpenTelemetry → New Relic, com traces distribuídos (o contexto viaja dentro das mensagens), métricas de negócio, logs JSON com correlation id e dashboard provisionado por Terraform.

4. Saga Pattern — estratégia escolhida: orquestração

A Saga coordena a transação distribuída *abrir OS → orçar → aprovar → pagar → executar → finalizar*, que passa por três bancos diferentes. O orquestrador é uma state machine do MassTransit dentro do OS Service (`OrdemServicoStateMachine.cs`), com o estado persistido na tabela `saga_ordem_servico`.



Rollback e compensação em cada etapa

Falha	Compensação comandada pelo orquestrador
Billing não consegue gerar o orçamento	A OS permanece em diagnóstico, com a falha registrada no histórico; o técnico pode reenviar.
Cliente reprova, ou staff cancela antes do pagamento	`CancelarOrcamento` no Billing.
Pagamento recusado pelo Mercado Pago	`CancelarOrcamento` e OS cancelada, com o motivo.
Falha durante a execução (ex.: peça indisponível)	`EstornarPagamento` (refund no Mercado Pago); a OS só é cancelada após a confirmação do estorno.
Staff cancela a OS em execução	`CancelarExecucao` + `EstornarPagamento`.

Garantias de consistência

- **Outbox transacional**: a mudança na OS, o estado da Saga e as mensagens a publicar são gravados no mesmo `SaveChanges`. Uma queda entre gravar e publicar não perde nem duplica mensagens.
- **Idempotência**: o RabbitMQ entrega "pelo menos uma vez"; os consumers e as ações da Saga são idempotentes por `OrdemServicoId`, e o inbox deduplica as mensagens recebidas.
- **Retry**: três novas tentativas com intervalo; esgotadas, a mensagem vai para a fila `_error`.

Por que orquestração e não coreografia

- O fluxo da OS é **sequencial e tem um dono natural** (o OS Service, que já era o dono do status). Com coreografia, a sequência ficaria implícita, espalhada em handlers de três repositórios.
- **Compensação centralizada**: regras como "se a execução falhar, estorne e só depois cancele" ficam legíveis num único arquivo.
- **Participantes simples**: Billing e Execution só recebem um comando e publicam o resultado, sem conhecer o fluxo.
- **Observabilidade**: o estado da Saga de cada OS é consultável (`GET /api/ordens-servico/{id}/saga`) e o trace distribuído mostra a cadeia completa.
- **Custo aceito**: o OS Service vira ponto central do fluxo. Mitigado porque as mensagens ficam enfileiradas no RabbitMQ enquanto ele está fora e a Saga retoma de onde parou.

5. Justificativa da divisão dos microsserviços

- **Fronteiras por capacidade de negócio**, seguindo as áreas da oficina citadas no desafio: atendimento (OS), cobrança (Billing) e produção (Execution). Cada uma muda por motivos diferentes e escala de forma diferente: a fila de execução tem picos próprios, e o pagamento depende de um terceiro (Mercado Pago).
- **Isolamento de falhas**: uma indisponibilidade do Mercado Pago afeta só o Billing; as mensagens esperam no RabbitMQ e o resto do sistema segue atendendo.
- **Dono único de cada dado**: o status oficial da OS é do OS Service; orçamento e pagamento, do Billing; o andamento do reparo, do Execution. Nenhum serviço lê o banco de outro.

6. Tecnologias e justificativas

Tecnologia	Uso e justificativa
.NET 8 (C#)	Mesma base das fases anteriores: LTS, alto desempenho, ecossistema maduro para APIs.
PostgreSQL (OS e Billing)	Dados transacionais e relacionais (OS ↔ cliente ↔ veículo; orçamento ↔ pagamento). O outbox precisa de transação ACID.
MongoDB (Execution)	A execução é um agregado autocontido (serviços, tempos e histórico aninhados), lido e gravado inteiro, sem joins; o esquema evolui por tipo de reparo. Atende ao requisito de banco NoSQL.
RabbitMQ + MassTransit 8	Broker leve, executado dentro do próprio cluster. O MassTransit entrega state machine de Saga, outbox/inbox, retry e propagação de trace prontos. Usa-se a versão 8 (licença Apache 2.0).
Mercado Pago (Checkout Pro)	Integração de pagamento exigida. Criação da preferência, webhook validado consultando a API do MP e estorno. Um gateway fake é usado em desenvolvimento e CI.
Kubernetes (k3s) + Traefik	Orquestração já usada nas fases anteriores; um Ingress por serviço, roteado por prefixo de caminho.
GitHub Actions	Pipeline independente por repositório: build → testes com gate de cobertura de 80% → SonarCloud → imagem no GHCR → deploy em cluster efêmero (kind) com smoke test → deploy em produção.
xUnit, Moq, Reqnroll (BDD)	Testes unitários e de integração nos três serviços; BDD em Gherkin cobrindo o fluxo completo da Saga e as compensações.
SonarCloud	Validação de qualidade de código no CI de cada serviço.
New Relic + OpenTelemetry	Ferramenta de observabilidade da Fase 3, estendida com traces distribuídos entre os serviços e um painel dedicado à Saga.

7. Testes e qualidade

Serviço	Testes automatizados	Cobertura de linhas
OS Service	374 (unitários, integração e 4 cenários BDD)	89%
Billing Service	48 (unitários e integração)	97%
Execution Service	30 (unitários e integração)	94%

O pipeline falha abaixo de 80% de cobertura. As branches principais dos três repositórios são protegidas: merge somente via Pull Request, com o check de build e testes obrigatório.